

Reviewer Pack — Minimal Geometric Entropy Correlation with Communication Ran...

Entry 21c5d6ea8feb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 08:10:23 UTC

Minimal Geometric Entropy Correlation with Communication Rank via Morse Theory

Entry ID: 21c5d6ea8feb **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 08:10:23 UTC

1. Conjecture

Field A (mathematical branch): Morse Theory **Field B** (complexity object): Communication Complexity

Statement:

For every boolean function f with n variables, the communication rank of its associated product-form communication protocol $r(f)$ is linearly correlated with the topological entropy $h(\text{Morse}(f))$, such that $r(f) = \Theta(h(\text{Morse}(f)))$ and $|h(\text{Morse}(f)) - r(f)| \leq 2^{n/3}$.

Rationale (proposer's reasoning):

Morse Theory provides a way to study the topology of real-valued functions by analyzing their critical points. The topological entropy $h(\text{Morse}(f))$ measures the complexity of the critical set, which may reflect the difficulty of communicating f . Communication rank $r(f)$ quantifies the communication complexity of f . If higher Morse entropy correlates with higher communication rank, it could suggest a deeper connection between these mathematical areas.

Taxonomy category: MORSE_THEORY_COMMUNICATION_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6bf257cc2fbed78f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between topological entropy $h(\text{Morse}(f))$ and communication rank $r(f)$ for all boolean functions f with n variables is linear, with an absolute error of at most $2^{n/3}$ across 30 random seeds. The criterion is falsified if any seed produces a correlation coefficient outside this range.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Morse Theory" AND "Communication Complexity" AND communication rank"
- "topological entropy" IN "product-form communication protocol" AND boolean function" -
"communication rank" ~ " $\theta(\text{topological entropy})$ " AND Morse theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def morse_function(f):
        n = int(math.log2(len(f)))
        morse_f = []
        for i in range(2**n):
            count_0 = f[i].count(0)
            count_1 = f[i].count(1)
            if count_0 == 0 or count_1 == 0:
                morse_f.append(0)
            else:
                morse_f.append(count_1 / (count_0 + count_1))
        return morse_f

    def topological_entropy(h):
        h = [x for x in h if x != 0]
        return sum(x * math.log2(1 / x) for x in h)

    def communication_rank(morse_f):
        n = int(math.log2(len(morse_f)))
        rank = 0
        for i in range(n):
            count_0 = morse_f[i].count(0)
```

```

        count_1 = morse_f[i].count(1)
        if count_0 == 0 or count_1 == 0:
            continue
        rank += max(count_0, count_1)
    return rank

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    f = generate_boolean_function(n)
    morse_f = morse_function(f)
    h_morse_f = topological_entropy(morse_f)
    r_f = communication_rank(morse_f)
    results.append((n, h_morse_f, r_f))

if not results:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "empty_results"
    }

n_max = max(n for n, _, _ in results)
instances_tested = len(results)

correlation_coefficients = []
for i in range(instances_tested):
    for j in range(i + 1, instances_tested):
        h_i, r_i = results[i][1], results[j][2]
        h_j, r_j = results[j][1], results[i][2]
        numerator = (h_i - h_j) * (r_i - r_j)
        denominator = math.sqrt((h_i**2 + h_j**2) * (r_i**2 + r_j**2))
        if denominator == 0:
            correlation_coefficients.append(0)
        else:
            correlation_coefficients.append(numerator / denominator)

mean_corr = sum(correlation_coefficients) / len(correlation_coefficients)
std_corr = math.sqrt(sum((x - mean_corr)**2 for x in correlation_coefficients) / len(correlation_coefficients))

conjecture_holds = all(abs(corr - 1) <= 2*n_max/3 for n, h_morse_f, r_f in results)
counterexample = "" if conjecture_holds else "correlation_outside_bound"

return {
    "metric_name": "correlation_coefficient",
    "metric_value": mean_corr,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\"}, \"metric_value\": {result['metric_value']}}")
        results.append(result)

    mean_corr = sum(r["metric_value"] for r in results) / len(results)
    std_corr = math.sqrt(sum((r["metric_value"] - mean_corr)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_corr} std={std_corr} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_outside_bound\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3b71c48e.py", line 107, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3b71c48e.py", line 56, in run_trial
    morse_f = morse_function(f)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3b71c48e.py", line 28, in morse_function
    count_0 = f[i].count(0)
              ~~~~~^~~~~~
AttributeError: 'int' object has no attribute 'count'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the correlation coefficient between topological entropy and communication rank as required by

the conjecture. | next: Investigate the cause of the crash in the test code and attempt to run the test again to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14148
2	preregistration	ollama_remote	glm4:latest	0	0	9512
3	novelty	ollama_remote	glm4:latest	0	0	8558
4	novelty	ollama_remote	glm4:latest	0	0	21404
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24650
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12276
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16890
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14897
9	judge	ollama_remote	glm4:latest	0	0	8888

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 131222 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/21c5d6ea8feb.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/21c5d6ea8feb.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/21c5d6ea8feb.tar.gz* (if generated)